Speech script: line by line

Use it as scaffolding, not as a script to read on the day. Read it aloud a few days running until the sentences feel like your own, then drop it and speak from the slide and its takeaway bar.

How to use it over a few days

- **Day 1:** read the whole thing aloud, standing, twice.
- **Day 2:** read it again, then cover each slide's text and say the *first and last line* from memory.
- **Day 3:** look only at the slide titles in the deck and say each section in your own words. Check against this script.
- **Day 4 onward:** no script. Use the one-page cue card in `PRESENTER_GUIDE.md`.

Marks used here: *[pause]* means stop for two seconds. *[point]* means point at the slide. **Bold** numbers are the ones to say slowly and clearly.

Say these words clearly: ARIMA is "uh-REE-muh". Diebold-Mariano is "DEE-bold mah-ree-AH-no". Bonferroni is "bon-feh-ROH-nee". PM2.5 is "P M two point five". µg/m³ is "micrograms per cubic metre". rMAE is "R M A E". GRU is "G R U"; LSTM is "L S T M".

Time: the script is about 3,800 words, which is 27 to 30 minutes at a calm pace. That leaves 3 to 5 minutes inside the 33-minute plan for pauses, pointing and changing slides. If you finish early, that is fine; never speed up to fit.

Slide 1. Title (about 30 seconds)

Good morning, everyone. *[pause]*
My project asks one question: how well can machine learning forecast hazardous air quality days in Delhi?
The answer I will build to is on the right of this slide. *[point]*
The number of alarms a health authority is willing to raise matters many times more than which model produces the forecast.
Over the next half hour I will take you through the data, the models, my three research questions, and what I would and would not claim.

Slide 2. Why this matters (about 1 minute 15 seconds)

Why does this matter?
A forecast is only useful if it arrives before the hazardous day does.
Fine particles called PM2.5 are linked to deaths from heart and lung disease, and to higher death rates overall.
That is why I forecast a health threshold, not a round number.
The threshold is **121 micrograms per cubic metre**. It is the boundary of the "Very Poor" category in India's national air quality index.
At the monitor I use, **29.7 percent** of days are above it. That is 920 days out of about three thousand.
Two earlier studies on Delhi used machine learning for PM2.5. They judged their forecasts by average error.

Neither judged them as a warning system with a stated number of alarms.
That is the gap I work on.
And one personal note: I grew up in India. For me, this is not an abstract statistic.

Slide 3. Three research questions (about 1 minute 15 seconds)

I asked three questions.
One: how accurately can machine learning forecast the concentration one to three days ahead, compared with classical and simple benchmarks?
Two: is it better to forecast the concentration and then apply a threshold, or to classify the hazardous day directly?
Three: judged as a warning system, what hit rate and false alarm rate does each approach give, and how does the decision threshold trade them off?
Here are the answers, so you know where we are going. *[point]*
One: the classical model, ARIMA, beats every machine learning regressor at every horizon. The deep networks beat it only at one day ahead.
Two: neither approach dominates, but forecast-then-threshold has a small edge, and I prefer it.
Three: the alarm budget moves the hit rate by about **fifty points**. The choice of model moves it by about **five**.
The third answer turned out to dominate the other two, so the talk is built around it.

Slide 4. The idea in one picture (about 45 seconds)

Here is the whole idea in one picture.
Think of a smoke alarm. You can change two things: which detector you buy, and how sensitive you set the dial.
On the left, I keep the budget fixed at one hundred alarms a year and swap the forecasting model. The hit rate stays between **0.82 and 0.87**. *[point]*
On the right, I keep the best model and turn the dial, from forty alarms a year to a hundred. The hit rate goes from **0.38 to 0.88**. *[point]*
Same vertical scale.
The detector matters little. The dial matters a lot. *[pause]*
The rest of the talk is the evidence for this picture, and the care I took not to fool myself.

Slide 5. The route (about 30 seconds)

Here is the route.
About seven minutes on the data, five on the models and how I test them, eleven on the results, then the limits and the conclusions.
And a question to hold in mind: how much does the model matter, compared with the number of alarms you allow?
A note on colour: blue is data and models, navy is ARIMA, the benchmark, and vermillion marks the number to remember.

Slide 6. The data (about 1 minute 15 seconds)

The data come from OpenAQ.
I compared five stations within 25 kilometres of central Delhi, on coverage rather than on how long their records claim to be.
I found something unexpected. Every station run by the Central Pollution Control Board returns data only from 19 February 2025. That is about eighteen months, even though the metadata says 2016.
Only one location, number 8118, returns the full record: almost ten years of daily values. So I used it.
There is a cost, and I want to state it plainly.
This monitor is an AirNow feed, almost certainly a United States diplomatic post.
So everything I show describes one well-characterised site in Delhi, not the whole city.
It gives me **3,093** usable days, and 29.7 percent of them are above the threshold.

Slide 7. Two defects in the data (about 1 minute 30 seconds)

Before any modelling, I found two problems in this data.
The first is a clipped ceiling. The value **1990** appears three times, identically, on unrelated dates.
One of them is on the second of September, in the monsoon, between days of 42 and 178.
That is not a measurement. It is an instrument limit. There were also five negative values.
The second problem is more dangerous, because it is invisible.
The platform measures how complete each day is by dividing the number of readings by 24, as if the sensor reported hourly.
But this sensor reported every half hour until 2024.
So a day with 24 of its 48 readings looked fully complete. Half a day of a polluted afternoon could enter the series as a daily average.
Days with only 50 to 75 percent of their true readings gave implausible values about **seven times** more often than days above 90 percent.
I fixed it by taking the reporting rate from the data itself.
In total I removed 282 of 3,375 days.
And the share of hazardous days barely moved, from 30.0 to 27.0 percent, so the filter is not removing the events I care about.

Slide 8. The series (about 1 minute)

This is the cleaned series, with hazardous days shaded. *[point]*
Notice that they come in winter episodes, not as isolated days.
That matters, because it makes simply repeating yesterday's value a strong forecast, as we will see.
The data are also strongly skewed. A few extreme days pull the average well above the median. The skewness is **2.17**.
So every model is fitted to the logarithm of the concentration.
Annual seasonal terms alone explain 69 percent of the variance of the log.

Slide 9. The calendar (about 30 seconds)

Here is the same data as a calendar. Each cell is a month, and darker means more hazardous days. *[point]*
The pattern repeats every year. October to February is dark, and the monsoon months are almost clear.
That regularity is the annual cycle the models carry.
It is also why a forecast that uses only the calendar is a strong baseline.

Slide 10. No weekly cycle (about 1 minute 30 seconds)

My original proposal assumed there was also a weekly cycle, caused by traffic.
It treated two seasonal periods as the central difficulty. I tested that assumption three ways.
First, the periodogram. Its strongest peaks are the annual cycle and its harmonics. Nothing sits near seven days. *[point]*
Second, autocorrelation. At lag seven, it is **0.173**. But lag six is 0.179 and lag eight is 0.153. So lag seven lies on the smooth decay. It is not a spike above it.
Third, day of the week. The spread between the highest and lowest day is 6.5 percent.
An analysis of variance gives p equals **0.34**. A rank test, which does not assume normal errors, gives p equals **0.77**.
Three tests, with different assumptions, all agree: there is no weekly cycle here.
There is a physical reason. A weekly rhythm is a traffic signature, and this monitor is in a diplomatic enclave, not beside a busy road.
So the model carries the annual cycle only. A simpler model, justified by the data.

Slide 11. The annual cycle and ARIMA (about 1 minute 30 seconds)

The annual cycle is the opposite. It is unmistakable: high in winter, low in the monsoon. *[point]*
So my benchmark is ARIMA on the log series. ARIMA predicts from recent values and recent errors.
I add four pairs of annual sine and cosine terms to carry the season, and no weekly term. *[point at the equation]*
I chose the order by an information criterion, over a grid of 24 candidates.
The order one-one-one scores **1892.5**. The next best scores 1893.6.
That gap of 1.1 is too small to call, so I take the simpler model.
The fitted coefficients are 0.581 and minus 0.987.
One honest detail. The second coefficient sits only 0.013 inside the boundary of validity.
In the rolling refits it crosses that boundary at three of 28 refits.
The optimiser is not constrained, so a forecast is still produced and scored. The details are in a backup slide.

Slide 12. The twelve models (about 45 seconds)

In total I compare twelve model specifications, from three families.
The classical and simple ones: persistence, which just repeats today; climatology, which uses only the calendar; a seasonal naive; and ARIMA.
Machine learning: ridge regression, random forest, and gradient boosting, the trees each fitted to the level and to the change from today.
Deep learning: a GRU, an LSTM and a transformer, each repeated over ten random seeds.

My supervisor asked me to try machine learning rather than rely on ARIMA. ARIMA stays as the reference point. Every model sees only the past, and every model is scored by the same code.

Slide 13. The three regressors (about 1 minute 15 seconds)

Three regressors, each controlling variance in a different way.
Ridge regression shrinks the coefficients. It has a closed form. *[point]*
The matrix X-transpose-X plus lambda times the identity is always invertible when lambda is positive. So a unique solution exists, even though lagged features are strongly correlated.
A random forest averages many trees. The variance of that average has a floor, rho times sigma squared, where rho is the correlation between trees.
Choosing random subsets of features at each split lowers rho, and so lowers the floor.
Gradient boosting builds trees one after another. Each tree fits the pseudo-residual, which is the negative gradient of the loss. It is gradient descent in function space.
I fitted the trees both to the level and to the change from today, because a tree cannot extrapolate beyond what it has seen, and this series drifts across the decade.

Slide 14. The three networks (about 1 minute 15 seconds)

Three neural networks, under one protocol, so that the only difference is the type of cell.
The GRU has two gates and no separate memory. The LSTM has three gates and an explicit memory cell. The transformer drops recurrence and uses attention across the window.
All three read the same 30-day window, with six channels.
They predict the change in the log concentration, trained with Adam on absolute error, with early stopping.
Each is run over ten seeds, and I report the range, never one favourable seed.
The parameter counts are 2,329, 3,097 and 3,409. A hybrid I tried as a bounded check has 3,288.
The LSTM and the transformer were added after my supervisor's feedback challenged two of my arguments.
So I tested them, instead of assuming.

Slide 15. The rolling origin (about 1 minute 15 seconds)

How do I test fairly? With a rolling origin.
Every model is scored at **2,303** forecast origins, over 6.8 years, after a three-year burn-in.
Parameters are refitted every 90 days, and every 180 days for the networks.
The rule is simple. At each origin, a model may use data up to that day, and nothing else.
But nothing fails visibly when that rule is broken. The numbers just get better.
So I test it directly. I corrupt the series from a chosen date, rerun everything, and require that no earlier forecast changes.
Why not a single train-test split? *[point at the box]*
On one calm block, ARIMA scored **0.903** at one day. Under the rolling harness, the same model scores **0.940**.
The single block was calmer than the training period, and it flattered every model.

Slide 16. The measures (about 1 minute)

Now the measures.
For accuracy I use relative mean absolute error, rMAE. It is the model's error divided by the error of simply repeating yesterday's value, on the same days.
So 1.0 is persistence, and below 1 is better.
For the warning system, I use three things.
The hit rate: the share of hazardous days that were caught.
The false alarm rate: the share of quiet days wrongly flagged.
And alarms per year, which is the unit an operator budgets in.
I never report plain accuracy. Hazardous days are a minority, so a model that never raises an alarm would score high.

Slide 17. Research question 1 (about 1 minute 45 seconds)

Now the results. Research question one.
This plot shows my accuracy table. From left to right: one, two and three days ahead.
Lower is better, and persistence is the dashed line at one. *[point]*
The three networks are bars, because each is a range over ten seeds.
At one day ahead, the networks run from **0.910 to 0.939**. ARIMA scores **0.940**. So the networks win, though by small margins.
At two days, it is closer.
At three days, the best network seed scores **0.831**, and ARIMA scores **0.832**. That is a tie. And the worst network seed is clearly weaker.
Now look at the grey dots. All five machine learning regressors sit behind ARIMA, at every horizon. *[point]*
That fits the literature. In the M4 forecasting competition, flexible learners did not reliably beat classical methods on a few thousand observations.
And my series has about three thousand.
So the answer to question one is: the classical model wins or ties, except at one day ahead, where the networks have a small edge.

Slide 18. One winter (about 30 seconds)

To make this concrete, here is one winter, November 2024 to February 2025.
Both ARIMA and the GRU follow the shape of each episode, not just its level.
The gap between them shows mainly on the sharpest rises, which is where a short-range model should help.
The GRU line is one seed, for illustration only. Every number I quote uses all ten.

Slide 19. Ranges, not numbers (about 1 minute)

This is why I report ranges.
For the GRU, the spread across ten seeds is **0.029** at one day, 0.028 at two, and **0.048** at three days.
At three days, that is wider than the gap between ARIMA and two other models.
So a single seed could be reported as beating them, or losing to them, at the analyst's convenience.

Even tree models moved by up to 0.004 between two machines, from the order of floating-point sums.
And one more thing. Rerunning the same seed on the same machine moved the GRU almost as much as a new seed.
So its training is not exactly reproducible. That is why I doubled the sample, from five seeds to ten.

Slide 20. Is it significant? (about 1 minute)

Are the one-day wins statistically real?
I used the Diebold-Mariano test against ARIMA, with a small-sample correction, once for each seed.
At one day, the GRU is ahead in all ten seeds, but the difference is significant in only **four**. For the LSTM it is **five**.
For the transformer, **seven**. *[point at the table]*
At two and three days, essentially no network reaches significance.
For the five machine learning regressors, all fifteen comparisons favour ARIMA, and fourteen are significant.
Now a limitation I want to state myself.
I ran 27 tests, without correction. After a Bonferroni correction, only six survive, and none of them is the plain GRU, LSTM or transformer.
So I read the results beyond one day as suggestive, not confirmed.

Slide 21. Two arguments I tested (about 1 minute)

In my literature review I argued for the GRU on two grounds. It has fewer parameters. And a transformer needs more data than I have.
On my supervisor's feedback, I tested both.
The LSTM beats the GRU at every horizon, by 0.005, 0.008 and 0.013, even though it has a third more parameters.
So the smaller network is not winning because it is smaller.
My supervisor pointed out that, counting every evaluation block, the series is about four thousand days.
The ten-seed run bears him out. The transformer beats the GRU in eight of ten matched seeds, at every horizon, and has the strongest significance at one day.
One caveat: the transformer's settings were inherited, not tuned. It is one configuration's result.
The lesson: a reasonable prior is not evidence.

Slide 22. Does weather help? (about 1 minute 15 seconds)

Does adding weather change the answer?
I added observed wind speed and temperature. A calm, stagnant air mass is a physical cause of episodes, and the calendar-only terms cannot see it.
At one day ahead, the weather network beats ARIMA in all ten seeds, and seven are significant.
At two days: three seeds win, six tie, one loses. At three days: three win, two tie, five lose.
Gradient boosting improves everywhere. The random forest gets worse at three days. So weather is not a free improvement.
And an important point. Under my first five seeds, two and three days both looked like wins. Doubling the seeds overturned that.
I also checked the weather record, which ends 202 days before the PM2.5 series.

Restricting to the fully observed window gives the same picture.

Slide 23. Weather and the alarm budget (about 30 seconds)

Does that accuracy gain matter for warnings? Barely.
I reran the threshold sweep with the weather models added. At several budgets they are not even the best option.
Where they do win, the gain is at most **1.2 points** of hit rate.
Gradient boosting improved its error by 8.2 points at one day. That became at most a little over one point of hit rate.
So the ceiling is not about the models being too inaccurate.

Slide 24. Research question 2 (about 1 minute 15 seconds)

Research question two. Forecast then threshold, or classify directly?
To compare fairly, I matched each classifier to a regressor built on the same algorithm, so the training objective is the only difference.
Across three pairs, three horizons and four budgets, that is 36 comparisons.
Forecast-then-threshold wins **21**. Classification wins **11**. Four are ties. *[point]*
The gap is small: a median of 0.011 in hit rate.
The edge shrinks as the budget loosens. At a budget of forty alarms, forecast-then-threshold wins 8 of 9. At a hundred alarms, classification wins 6 of 9.
I prefer forecast-then-threshold for one more reason.
It gives a concentration, which can be rethresholded for any future health policy without refitting.
A classifier is tied to the threshold it was trained on.

Slide 25. What an alarm budget means (about 1 minute)

Now research question three, the central result. First, what an alarm budget means.
Each grid here is 100 hazardous days. The orange dots are the days the best model caught, at one day ahead. *[point]*
At 40 alarms a year, about one alarm every nine days, it catches **38** of 100.
At 60 alarms: 57. At 80: 74.
At 100 alarms a year, about one every four days, it catches **88**.
Nothing changed except the dial.
This is a policy choice about how many alarms the public will tolerate. It is not a modelling choice.

Slide 26. Step one (about 30 seconds)

Now the chart behind it.
Horizontally, alarms raised per year: the dial. Vertically, the share of hazardous days caught.
This is one model, ARIMA. As the budget rises, so does the hit rate.
That is true by construction. So on its own it says nothing about models.
The question is what happens when I change the model.

Slide 27. Step two (about 30 seconds)

Now I add three very different forecasters.
Gradient boosting. Persistence, which only repeats yesterday. And climatology, which uses only the calendar.
Four very different forecasters, and almost one curve. *[pause]*
I have not separated them for legibility. The overlap is the finding.

Slide 28. The two effects (about 1 minute 15 seconds)

Now the two effects.
Along the dial, from 40 to 100 alarms a year, the best hit rate goes from **0.379 to 0.881**. That is fifty points. *[point]*
Hold the budget at 100 alarms and change the model, and the hit rate moves from about 0.82 to 0.88. *[point]*
My report summarises this as about five points. If I recompute it across all ten models, it averages seven.
Either way, it is many times smaller than the effect of the dial.
Two examples make it sharper.
At three days ahead and a budget of 40 alarms a year, persistence is the best model in the whole study, at **0.359**.
It beats every machine learning model, ARIMA, and every network seed.
And at three days with 100 alarms, a climatological forecast, using no recent information at all, comes within a hundredth of the best: 0.816 against 0.827.
The reason is that hazardous days come in multi-day episodes.
The most informative fact about whether day three is hazardous is often just that today already is.
So: the dial matters more than the detector. *[pause]*

Slide 29. What I checked (about 1 minute)

What did I check, and what did it change?
The cleaning ceiling of 1000 is a judgement. So I restored the seven removed days and refitted everything.
ARIMA barely moves: 0.935 against 0.940. But the machine learning models get worse.
Ridge goes from a near tie with persistence to a clear loss: 0.997 to 1.118. So question one strengthens.
I also ran bounded checks on the GRU's own settings.
At one and two days, the default is near the best. At three days it is the setting I can least defend.
A 45-day window, four seasonal pairs, or a different optimiser each improve it by two to four points.
These checks used one split and three seeds. I did not rerun the full evaluation, so the main table is unchanged.

Slide 30. What the results do not show (about 45 seconds)

What do the results not show?
One site, in one city, in a diplomatic enclave.
Observed weather, not forecast weather.
The LSTM and transformer were not given weather or the threshold sweep.
Tree grids were small, so the machine learning results are a lower bound.
The significance tests were not jointly corrected, and I picked best seeds.
And network training is not exactly reproducible.

None of these overturns the central finding, because it rests on several independent sources agreeing, not on a single p-value.

Slide 31. What the project contributes (about 1 minute)

So what does this project contribute? Four things.
One: a warning-system evaluation, with hit rate, false alarm rate and alarms per year under an explicit budget. The Delhi studies I found do not report that.
Two: a leakage-tested comparison of twelve specifications, from classical to deep, with networks reported as ten-seed ranges.
Three: two defects in the source data, diagnosed and corrected before any model was fitted.
Four: priors tested rather than assumed. No weekly cycle. The GRU is not better because it is smaller. And the small-data argument against a transformer fails.
Applying standard models to Delhi is not new.
Judging them as a warning system, with a varying budget, is.

Slide 32. Three questions, three answers (about 1 minute 15 seconds)

To conclude: three questions, three answers.
One: ARIMA beats every machine learning regressor at every horizon, with a test behind each comparison. The networks beat it only at one day ahead, as ranges over ten seeds.
Two: neither approach dominates. Forecast-then-threshold wins 21 of 36 matched comparisons, 11 go to classification, and four are ties. I prefer forecast-then-threshold, because a concentration can be rethresholded.
Three: the budget moves the hit rate by fifty points. The model moves it by a few.
Along the way, I corrected two data defects, and I tested an assumption from my own proposal, a weekly cycle, and found it false.
My supervisor asked me to try several methods rather than rely on ARIMA. He was right.
It is what makes the negative result at two and three days credible, and what surfaces the one-day result.

Slide 33. The recommendation (about 1 minute)

My recommendation is this. *[slow]*
Design a hazardous air quality warning system around the alarm budget its operator can justify, and treat the forecasting model as a secondary refinement.
A model comparison that does not fix the alarm budget it was evaluated at has answered a question nobody who operates the system asked.
Six extensions follow from the report.
Forecast weather in place of observed weather. Boundary layer height from a reanalysis such as ERA5. The GRU with four seasonal pairs under the full ten-seed evaluation.
A second, independent station. Weather channels and the threshold sweep for the LSTM and the transformer.
And a full ten-seed evaluation of the Kolmogorov-Arnold hybrid.
Thank you. I am happy to take questions. *[smile, and wait]*

Slide 34. Questions

[Take a breath. Listen to the whole question. Pause. Answer in one sentence, give one number, state one limit. Then stop.]

Last, a reminder of what makes this good

- You do not need to be perfect. You need to be **clear, honest and calm**.
- Say the **takeaway bar** of every slide in your own words. If you remember nothing else, remember that.
- If you lose your place, look at the slide title, say it, and carry on. No one knows the order you planned.